

Corso Completo di Git & GitHub

Docente: **Moussa Bakabi Salisou**

Questa guida è stata strutturata per accompagnare lo studente dall'installazione e configurazione iniziale di Git fino alla gestione avanzata dei rami (branch), risoluzione dei conflitti e sincronizzazione sicura tramite protocollo SSH con piattaforme di hosting remoto come GitHub e GitLab.

Modulo 1: Configurazione Iniziale dell'Ambiente

Subito dopo aver installato Git, è fondamentale impostare l'identità globale del sistema. Queste informazioni verranno collegate in modo permanente a ogni commit effettuato, permettendo la tracciabilità delle modifiche all'interno del team di sviluppo.

1.1 Impostazione di Nome ed Email

Eseguire i seguenti comandi nel terminale per configurare i metadati globali:

```
git config --global user.name "Moussa Bakabi Salisou"  
git config --global user.email "la-tua-email@esempio.com"
```

Nota sul flag --global

Il flag **--global** indica a Git di utilizzare queste credenziali per tutti i repository presenti sul computer corrente. Se si desidera utilizzare un'identità differente per un progetto specifico, è sufficiente omettere il flag eseguendo il comando dall'interno della cartella del progetto interessato.

1.2 Verifica delle impostazioni

Per ispezionare l'elenco completo delle configurazioni attive e assicurarsi che i dati inseriti siano corretti, utilizzare il comando:

```
git config --list
```

Modulo 2: Autenticazione Sicura tramite Chiavi SSH

L'utilizzo del protocollo SSH (Secure Shell) elimina la necessità di inserire manualmente le proprie credenziali a ogni operazione di invio o ricezione del codice verso server remoti, garantendo al contempo un elevato standard di sicurezza crittografica.

2.1 Generazione della coppia di chiavi

Per generare una nuova coppia di chiavi SSH basata sul moderno algoritmo **ed25519**, eseguire il comando inserendo il proprio indirizzo email:

```
ssh-keygen -t ed25519 -C "la-tua-email@esempio.com"
```

Premere *Invio* a tutte le richieste successive per accettare il percorso di salvataggio predefinito e lasciare la passphrase vuota (se si preferisce un accesso automatizzato senza password aggiuntive).

2.2 Estrazione e Copia della Chiave Pubblica

È necessario copiare il contenuto del file della chiave pubblica (**.pub**) per poterlo associare alla piattaforma remota. Comandi rapidi di copia in base al sistema operativo:

Sistema Operativo	Comando di Copia Automatica
Windows (Git Bash)	<code>clip < ~/.ssh/id_ed25519.pub</code>
macOS	<code>pbcopy < ~/.ssh/id_ed25519.pub</code>
Linux	<code>cat ~/.ssh/id_ed25519.pub</code> (selezionare manualmente il testo a schermo)

2.3 Associazione su GitHub

Accedere a GitHub, navigare in **Settings > SSH and GPG keys**, cliccare su **New SSH Key**, assegnare un titolo mnemonico al dispositivo e incollare il testo precedentemente copiato all'interno del campo **Key**. Confermare il salvataggio.

2.4 Verifica della Connessione

Per validare la corretta configurazione e il corretto scambio delle chiavi, avviare il test di connessione:

```
ssh -T git@github.com
```

In caso di successo, il server restituirà un messaggio di avvenuta autenticazione personalizzato con il proprio username.

Modulo 3: Il Flusso di Lavoro Quotidiano (The Git Workflow)

La gestione dei file con Git segue un ciclo standardizzato suddiviso in tre aree concettuali: la directory di lavoro (Working Directory), l'area di preparazione (Staging Area o Index) e il database locale (Local Repository).

1 Ispezione dello stato attuale (git status)

Verifica quali file sono stati modificati, quali sono stati eliminati e quali non sono ancora tracciati dal sistema di controllo versione.

2 Preparazione delle modifiche (git add)

Sposta le modifiche selezionate nella Staging Area. Per preparare un singolo file usare `git add nomefile.ext` ; per preparare l'intera struttura del progetto corrente usare il punto: `git add .`

3 Consolidamento locale (git commit)

Crea un punto di ripristino immutabile nella cronologia locale del repository. Il comando deve essere sempre corredato da un messaggio descrittivo chiaro: `git commit -m "Messaggio esplicativo"`

Tabella dei comandi fondamentali per la gestione quotidiana

Comando	Scopo e Descrizione Operativa
<code>git init</code>	Inizializza un nuovo repository locale vuoto nella cartella di lavoro corrente.
<code>git clone <url></code>	Effettua il download completo di un repository esistente da un server remoto.
<code>git log --oneline</code>	Mostra la cronologia sintetica dei commit eseguiti sul branch corrente.
<code>git diff</code>	Mostra le differenze testuali riga per riga non ancora inviate alla staging area.

Modulo 4: Gestione Avanzata dei Branch e Sincronizzazione Remota

I rami (branch) consentono lo sviluppo parallelo di nuove funzionalità o la risoluzione di bug isolando il codice rispetto alla linea di produzione principale (solitamente denominata `main` o `master`).

4.1 Comandi di Branching Operativo

- `git branch` : Elenca tutti i rami locali indicando con un asterisco quello attualmente attivo.

- `git switch -c <nome-branch>` : Crea un nuovo ramo e vi si sposta immediatamente (equivalente al comando classico `git checkout -b`).
- `git switch <nome-branch>` : Consente di spostarsi da un ramo all'altro della Working Directory.

4.2 Integrazione delle Modifiche (Merge)

Per integrare i cambiamenti sviluppati su un ramo secondario all'interno del ramo principale, posizionarsi prima sul ramo ricevente (es. main) e lanciare il comando di unione:

```
git switch main
git merge nome-ramo-secondario
```

Gestione dei Conflitti di Merge

Se la stessa riga dello stesso file è stata modificata in modo differente su due rami separati, Git bloccherà il merge evidenziando il conflitto. Lo sviluppatore dovrà aprire il file interessato, rimuovere i marcatori di conflitto (`<<<<<<` , `=====` , `>>>>>>`), scegliere la porzione di codice corretta, aggiungere il file alla staging area con `git add` e concludere il processo eseguendo un normale `git commit` .

4.3 Allineamento con il Server Remoto

Per pubblicare i propri progressi o incorporare il lavoro svolto dal resto dei collaboratori, si utilizzano i comandi di rete:

```
# Invia il codice del branch locale al server remoto (origin)
git push origin nome-branch

# Scarica ed esegue il merge automatico delle novità remote sul codice locale
git pull
```